

IN4640 – Machine Vision

Assignment 02

Index No: 215565L

Name: Weerasinghe R.T.

GitHub Link: <https://github.com/rtweera/machine-vision-assignment-2>

1.

(a) Used Total Least Squares (TLS) to fit a line to the data corresponding to given first line points. Line is modelled in the standard $ax+by+c=0$ form. Idea is to minimize the orthogonal distances from the points to the line.

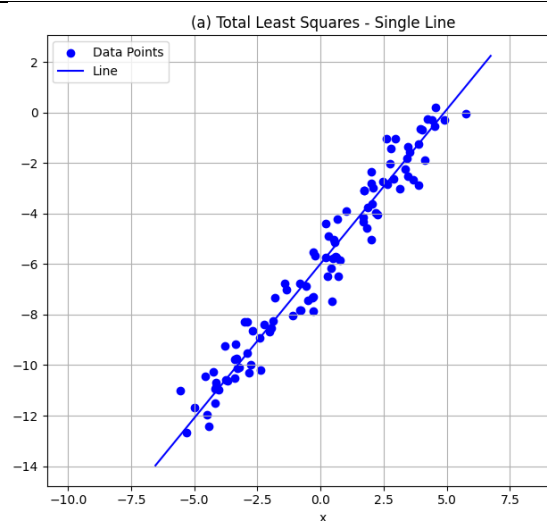
```
def fit_line_tls(x: np.ndarray, y: np.ndarray) -> np.ndarray:
    pts = np.column_stack([x, y])
    mu = pts.mean(axis=0)
    centered = pts - mu
    _, _, Vt = np.linalg.svd(centered)

    normal = Vt[-1]
    a, b = normal / np.hypot(*normal)
    c = -(a * mu[0] + b * mu[1])
    return np.array([a, b, c])
```

The algorithm centers the data by subtracting the mean and then applies Singular Value Decomposition (SVD) on the centered coordinates. In SVD, the last row of the transpose of V reflects the direction of least variance, meaning the normal of the best-fit line is that. This vector is then normalized so $a^2+b^2=1$, and the constant c is calculated using the mean point applying to $ax+by+c=0$. This gives following results:

```
=====
PART (a) - TLS (First Line)
=====
ax + by + c = 0 form: [ 0.77356165 -0.63372105 -3.79419221]
Slope form: y = 1.220666x + -5.987165
```

So,

a = 0.77**b = -0.63****c = -3.79**

(b) To fit all the three lines all points are used. Random Sample Consensus (RANSAC) is used to find the initial line. Then its inlier consensus is masked out, and the process is repeated to find the remaining lines.

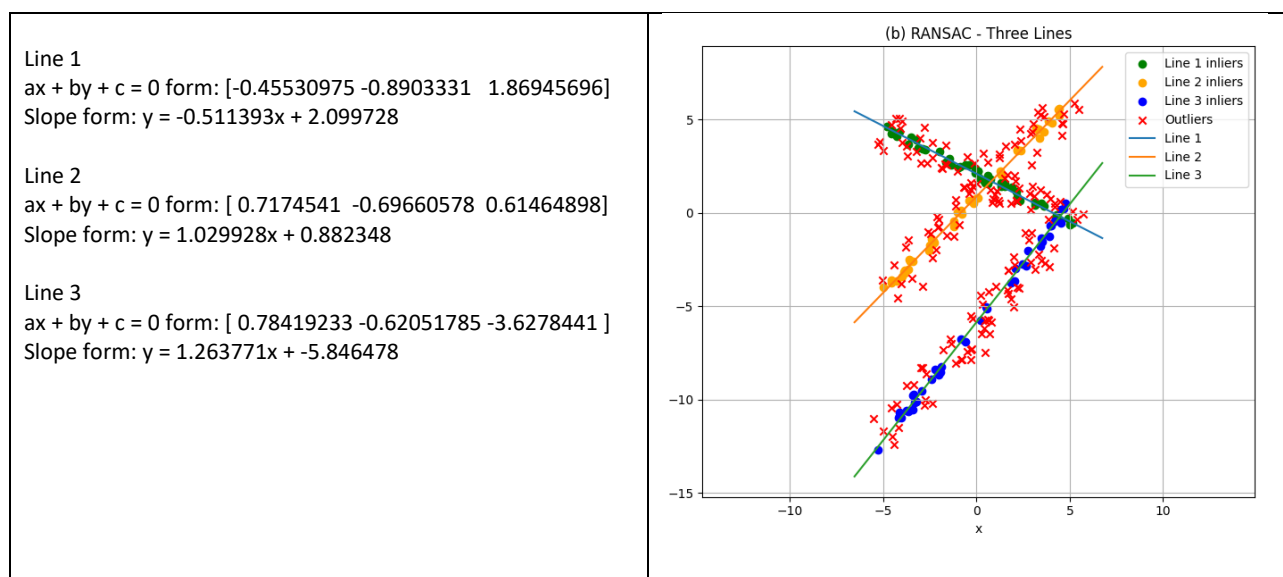
```
def ransac_fit_line(points: np.ndarray, n_iters=4000, threshold=0.25, min_inliers=35,
seed=0):
    # ... [Looping n_iters times to pick 2 random points] ...
    line = line_from_two_points(points[i], points[j])
    d = point_line_dist(line, points)
    inliers = d < threshold
    count = inliers.sum()

    if count > best_count and count >= min_inliers:
        best_count = count
        best_line = line
        best_inliers = inliers

    # Refine using TLS with the inlier points
    inlier_pts = points[best_inliers]
    refined = fit_line_tls(inlier_pts[:,0], inlier_pts[:,1])
    return refined, best_inliers
```

This selects two random points to define a proposed lines and calculates the orthogonal distance of all other points to this line. Points falling within a distance threshold (0.25) are classified as inliers. The line with the highest number of inliers is selected as the best fit.

Instead of just using RANSAC derived line from two points, I applied the TLS function on the filtered inliers to refine the line parameters for higher accuracy. The inliers are then masked and the process runs on the remaining points until all three lines are identified. Other visualization codes are also used for the below visualization, but the above is the primary RANSAC function.



Compared to a, b, c values in TLS (0.77, -0.63, -3.79) in part A, part B has (0.78, -0.62, -3.63) and showcases a better fitted line.

2.

This requires calculating the physical sizes of the flat earrings assuming the camera's optical axis is perpendicular to the imaging plane. Given parameters have focal length of 8mm, and a pixel size of 2.2um X 2.2 um.

NOTE: Problem states that the distance from the lens to the imaging plan (D') is 720mm, but it should be the distance from object to lens (D) because mathematically and practically 720mm D' doesn't make sense. With thin lens formula if $D'=720$ mm, D would be roughly 8.09mm which is too close. Also, if this is used, object height is too short (less than a millimetre). Hence, I assumed $D=720$ mm and D' is not given and calculated as follows:

```
def image_plane_distance_mm() -> float:
    """Compute D' from thin lens formula: 1/f = 1/D + 1/D'."""
    if OBJECT_DISTANCE_MM <= FOCAL_LENGTH_MM:
        raise ValueError("OBJECT_DISTANCE_MM must be greater than FOCAL_LENGTH_MM")
    return (FOCAL_LENGTH_MM * OBJECT_DISTANCE_MM) / (OBJECT_DISTANCE_MM -
FOCAL_LENGTH_MM)
```

First, we need to identify the earrings, so image is converted to HSV space to segment "gold-ish" non-white pixels. After some morphology to clean up noise, we make a binary mask. Then we do contour detection and align bounding boxes on the largest two contours. Largest 2 contours in this picture must be the two earrings.

```
def segment_earrings(bgr: np.ndarray) -> np.ndarray:
    """
    Return a binary mask of earring pixels.
    Strategy:
    - Segment non-white / "gold-ish" using HSV (higher saturation, not too bright)
    """
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    H, S, V = cv2.split(hsv)

    # Mask: not-white-ish pixels
    # - saturation > ~20 (out of 255) avoids pure whites/greys
    # - value < ~250 avoids very bright background
    mask = ((S > 20) & (V < 250)).astype(np.uint8) * 255

    # Clean up noise
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
    return mask

def find_two_earrings(mask: np.ndarray):
    """
    Find contours and return the two largest components' bounding boxes.
    Returns list of dicts with contour, bbox, area.
    bbox = (x, y, w, h)
    """
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    comps = []
```

```

for c in cnts:
    area = cv2.contourArea(c)
    if area < 200: # ignore tiny specks
        continue
    x, y, w, h = cv2.boundingRect(c)
    comps.append({"contour": c, "area": area, "bbox": (x, y, w, h)})

comps.sort(key=lambda d: d["area"], reverse=True)
return comps[:2] # largest = earring 1, second largest = earring 2

```

Now we have the height and width of these earrings in pixels. Pixel widths and heights are converted to mm by considering the pixel pitch. Knowing D' , D , and y' (height in sensor in mm) using similar triangles property, we can find real object height y as $y = y' * D / D'$.

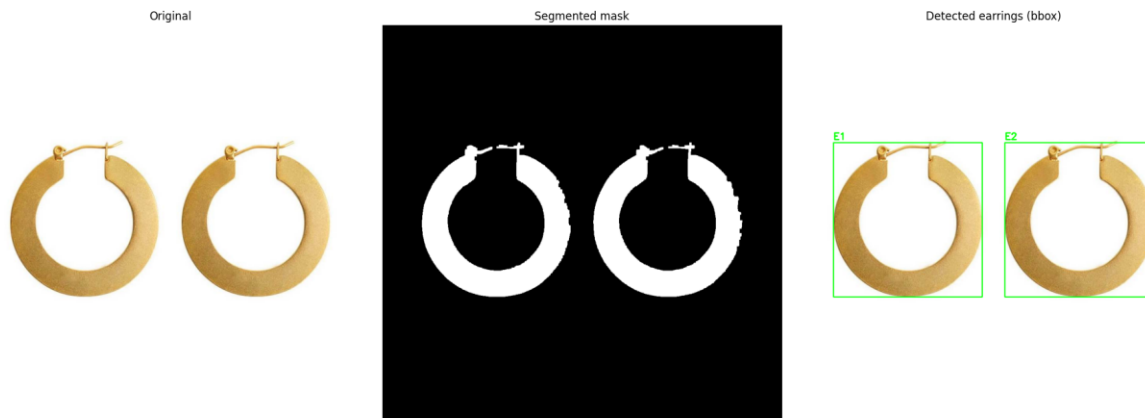
```

def sensor_to_object_mm(sensor_mm: float) -> float:
    """
    Convert sensor size  $y'$  (mm) to object size  $y$  (mm).

    Using similar triangles:  $y / y' = D / D'$ 
    Using thin lens:  $1/f = 1/D + 1/D' \Rightarrow D' = fD / (D - f)$ 
    """
    d_prime_mm = image_plane_distance_mm()
    return (OBJECT_DISTANCE_MM / d_prime_mm) * sensor_mm

```

Here's the results:



=== Measurements (bounding box based) ===

$f = 8.0$ mm, $D = 720.0$ mm, $D' = 8.0899$ mm, pixel = 2.2 μ m

Earring 1:

bbox pixels (w,h) = (381, 395)

sensor size (mm) = (0.8382, 0.8690)

object size (mm) = (74.60, 77.34)

Earring 2:

bbox pixels (w,h) = (382, 395)

sensor size (mm) = (0.8404, 0.8690)

object size (mm) = (74.80, 77.34)

Average earring size (mm) from the two boxes:

avg width = 74.70 mm

avg height = 77.34 mm

avg diameter = 76.02 mm

3. Goal is to superimpose a country flag on the cricket turf. Given code segments in the assignment was used to detect the mouse clicks and to find the edges of the turf. After collecting the clicked points, the following function was used to organize these points in the TL, TR, BR, BL order:

```
def order_points_by_angle(pts):
    """ Order 4 points in TL, TR, BR, BL order by sorting based on angle from
    centroid."""
    pts = np.array(pts, dtype=np.float32)
    c = pts.mean(axis=0)
    angles = np.arctan2(pts[:,1] - c[1], pts[:,0] - c[0])
    pts = pts[np.argsort(angles)] # CCW; consider centroid as origin and sort by
    angle asc to get TL, TR, BR, BL in CCW order
    return pts
```

We find the centroid by getting the mean of points, and then check the angles of the points. Based on the angles we sort the points in ascending order. This results in TL, TR, BR, BL order we want so the user doesn't have to click in correct order for this program to work.

To get the minimal output of just putting the flag on the turf, we first find the homographic transformation of flag image to the warped turf by making a rectangle of flag size and warping it to match the turf points. This gives us the homography H. Using that we can transform the actual flag now, and overlay it on the turf.

```
Ht, Wt = turf.shape[:2]
Hf, Wf = flag.shape[:2]
```

```

src = np.array([[0, 0],
               [Wf-1, 0],
               [Wf-1, Hf-1],
               [0, Hf-1]], dtype=np.float32)
# dst_quad is the points we clicked
H = cv2.getPerspectiveTransform(src, dst_quad) # 3x3 homography matrix to warp
flag onto the destination quad
warped_flag = cv2.warpPerspective(flag, H, (Wt, Ht), flags=cv2.INTER_LINEAR) #
interpolate for smoothness

```

Simple output:



But this looks fake. So to make it more realistic we can desaturate the image, blur the edges, apply transparency, try to replicate the texture of the turf on the flag by adjusting brightness and so on.

Getting a realistic image:

The below codes add a gaussian blur to the flag and also make a mask that allows us to blend the flag to the turf in a natural manner.

```

if gauss_blur and gauss_blur > 0: # blur to make it more like painted on the
surface
    k = gauss_blur if gauss_blur % 2 == 1 else gauss_blur + 1
    warped_flag = cv2.GaussianBlur(warped_flag, (k, k), 0)

# apply a mask to blend only in the warped area of the flag
white = np.ones((Hf, Wf), dtype=np.uint8) * 255
warped_mask = cv2.warpPerspective(white, H, (Wt, Ht), flags=cv2.INTER_NEAREST)
if feather and feather > 0:
    k = feather if feather % 2 == 1 else feather + 1
    warped_mask = cv2.GaussianBlur(warped_mask, (k, k), 0) # feather the edges
to make it natural
m = (warped_mask.astype(np.float32) / 255.0)[:, :, None] # normalize mask to
[0..1] and make it 3-channel (for mixing with color images)

```

Turf texture is imprinted on the flag by first finding the high frequency components in the turf, and using the positions of those components to modulate the flag's brightness values in those spots.

```

# imprint turf texture onto flag (makes it look painted/printed)
turf_gray = cv2.cvtColor(turf, cv2.COLOR_BGR2GRAY).astype(np.float32) / 255.0
turf_blur = cv2.GaussianBlur(turf_gray, (0, 0), 5)
texture = np.clip((turf_gray - turf_blur), -0.25, 0.25) # high-frequency
component ~ grass texture
warped_flag_f = warped_flag.astype(np.float32) / 255.0 # normalize to [0..1] for
blending
mod = (1.0 + texture_mix * texture)[:, :, None] # modulate value for brightness;
make 3 channel
warped_flag_f = np.clip(warped_flag_f * mod, 0, 1)

```

Finally, the mask `m`, texture imprinted flag `warped_flag_f` are used with an overall alpha value which controls how much blend is happening between the turf and the flag.

```

# final alpha blend only in masked region
turf_f = turf.astype(np.float32) / 255.0
out = turf_f * (1 - alpha*m) + warped_flag_f * (alpha*m) # apply mask to make
realistic.
out = np.clip(out * 255.0, 0, 255).astype(np.uint8)
return out

```

All these things are bundled to a function and is called with the following arguments.

```

result = overlay_flag_realistic(
    turf, flag, dst_quad,
    alpha=ALPHA, feather=FEATHER,
    texture_mix=TEXTURE_MIX, gauss_blur=GAUSS_BLUR
)

```

```

ALPHA = 0.30          # overall alpha blend; 0 = turf only, 1 = flag only
FEATHER = 31          # edge softness (odd number)
ROTATE_FLAG_K = 1     # 0=no rotate, 1=90° CW, 2=180, 3=270 CW

SAT_SCALE = 0.70      # <1 makes it less "neon"
VAL_SCALE = 0.80      # <1 slightly darker
TEXTURE_MIX = 1       # how much turf texture imprints on the flag [0..1]
GAUSS_BLUR = 5        # small blur on warped flag to blend into scene

```

Flag is also desaturated with the following function:

```

def make_flag_less_saturated(flag_bgr, sat_scale=0.85, val_scale=0.98):
    hsv = cv2.cvtColor(flag_bgr, cv2.COLOR_BGR2HSV).astype(np.float32)
    hsv[..., 1] = np.clip(hsv[..., 1] * sat_scale, 0, 255) # less neon by reducing
saturation; clip to valid range (just in case)
    hsv[..., 2] = np.clip(hsv[..., 2] * val_scale, 0, 255) # slightly darker by
reducing value; clip to valid range (just in case)
    return cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2BGR)

```

With this we receive the following outcome:



In my opinion, this is highly realistic (as in it looks painted on the turf, rather than digitally put on top of the turf), compared to the original simple result.

